ROS 2 JAZZY
LAB 1

FULL SOURCE CODE + DETAILED EXPLANATION

Turtlesim • Custom Interfaces • Actions • Services • Parameters • Launch

Ubuntu 24.04 | ROS 2 Jazzy | Python 3 | colcon

This edition is designed as a complete learning and submission companion. It explains the architecture, includes the full contents of the project source files, explains the important code blocks, provides terminal commands and test cases, and finishes with GitHub publishing and viva preparation.

Verification honesty: Python files are parsed for syntax and package XML files are checked for well-formedness during document creation. A live ROS 2 build and simulator test must still be run in your Ubuntu/WSL2 environment.

Contents

- 1. Project overview and deliverables
- 2. ROS 2 concepts
- 3. Architecture and execution flow
- 4. Workspace and package structure
- 5. Custom interfaces explained
- 6. Setup, build, and launch
- 7. Complete source code — GoToPose server
- 8. Complete source code — Reset service server
- 9. Complete source code — DrawSquare server
- 10. Launch file and parameters
- 11. How to test every requirement
- 12. Troubleshooting
- 13. Viva questions and answers
- 14. GitHub publishing
- 15. Final submission checklist
- Appendix A. Package metadata and README

1. Project overview and deliverables

The project controls the turtlesim turtle using ROS 2. The custom `TargetPose` message represents a target x/y position and yaw angle. The `GoToPose` action moves the turtle toward that target while reporting distance feedback. The `ResetTurtle` service optionally clears the trail and returns the turtle to the center. The `DrawSquare` action builds a square by sending exactly four GoToPose goals. The `SetSpeed` service changes maximum linear and angular speed at runtime.

Main deliverables

- Custom message: TargetPose.msg
- Actions: GoToPose.action and DrawSquare.action
- Services: ResetTurtle.srv and SetSpeed.srv
- Three Python node implementations
- Launch file and parameter YAML
- README, package metadata, and GitHub-ready structure

2. ROS 2 concepts

Concept	Meaning in this project
Node	A program/component in the ROS graph. The launch file starts turtlesim and three controller nodes.
Topic	A stream of messages. `/turtle1/pose` publishes position/orientation; `/turtle1/cmd_vel` accepts velocity commands.
Service	A request/response operation. `/reset_turtle` resets the turtle; `/set_speed` updates speed limits.
Action	A long-running operation with goal, feedback, and result. Used for `/goToPose` and `/draw_square`.
Parameter	A runtime setting such as max linear speed, max angular speed, or a tolerance.
Launch	A Python launch description starts multiple nodes with one command.

3. Architecture and execution flow

When the launch file runs, it starts turtlesim, the GoToPose action server, the ResetTurtle service server, and the DrawSquare action server. The square server resets the turtle, calculates four target corners, and sends four goals to GoToPose. The GoToPose server reads the current pose and publishes velocity commands. Feedback reports progress.

```
User / CLI
   |
   | goal: side_length, speed
   v
/draw_square action server
   |---- calls /reset_turtle service
   |---- sends GoToPose goal #1
   |---- sends GoToPose goal #2
   |---- sends GoToPose goal #3
   |---- sends GoToPose goal #4
   v
/goToPose action server
   |---- subscribes /turtle1/pose
   |---- publishes /turtle1/cmd_vel
   v
turtlesim window
```

4. Workspace and package structure

Keep the package directories directly inside `~/ros2_ws/src/`. `lab1_interfaces` uses `ament_cmake` because ROS generates code from the interface definitions. `lab1_controller` uses `ament_python` for Python executables.

```
ros2_lab1_github/
■■■ lab1_interfaces/
■   ■■■ msg/TargetPose.msg
■   ■■■ action/GoToPose.action
■   ■■■ action/DrawSquare.action
■   ■■■ srv/ResetTurtle.srv
■   ■■■ srv/SetSpeed.srv
■   ■■■ CMakeLists.txt
■   ■■■ package.xml
■■■ lab1_controller/
■   ■■■ lab1_controller/
■   ■   ■■■ go_to_pose_server.py
■   ■   ■■■ reset_server.py
■   ■   ■■■ square_server.py
■   ■■■ launch/lab1.launch.py
■   ■■■ config/params.yaml
■   ■■■ setup.py
■   ■■■ package.xml
■■■ README.md
■■■ LICENSE
■■■ docs/
```

5. Custom interfaces explained

TargetPose.msg

Contains `x`, `y`, and `yaw`. x/y are coordinates in the turtlesim window. yaw is the desired heading in radians.

Complete definition

```
float64 x
float64 y
float64 yaw
```

GoToPose.action

The goal contains a TargetPose. The result contains a Boolean success and a message. Feedback reports `distance_remaining` while the turtle moves.

Complete definition

```
lab1_interfaces/TargetPose target_pose
---
bool success
string message
---
float64 distance_remaining
```

DrawSquare.action

The goal contains side_length and speed. The result reports success and a message. Feedback reports the number of sides completed (0–4).

Complete definition

```
float64 side_length
float64 speed
---
bool success
string message
---
uint8 sides_completed
```

ResetTurtle.srv

The request contains clear_screen. The response contains success and a message.

Complete definition

```
bool clear_screen
---
bool success
string message
```

SetSpeed.srv

The request contains maximum linear and angular speed. The response confirms whether the values were accepted.

Complete definition

```
float64 max_linear_speed
float64 max_angular_speed
---
bool success
string message
```

6. Setup, build, and launch

Install the required packages if they are not already present:

Install prerequisites

```
sudo apt update
sudo apt install -y ros-jazzy-turtlesim python3-colcon-common-extensions python3-rosdep
```

Make sure both package folders are in `~/ros2_ws/src/`. Then run:

Build the workspace

```
source /opt/ros/jazzy/setup.bash
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

Launch all nodes in one terminal:

Launch the project

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch lab1_controller lab1.launch.py
```

Keep this terminal running. Open another terminal for test commands and source both setup files there as well.

7. Complete source code — GoToPose server

This server is the movement controller. It receives a TargetPose goal, calculates the position and heading error, publishes Twist commands, sends feedback, and returns a result. It also exposes `/set_speed` to update the speed parameters.

go_to_pose_server.py — complete file

```
#!/usr/bin/env python3
"""Action server that drives turtlesim to a requested pose."""
import math

import rclpy
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.node import Node
from rcl_interfaces.msg import SetParametersResult
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
from lab1_interfaces.action import GoToPose
from lab1_interfaces.srv import SetSpeed


def normalize_angle(angle: float) -> float:
    """Normalize an angle to [-pi, pi]."""
    return math.atan2(math.sin(angle), math.cos(angle))


class GoToPoseServer(Node):
    def __init__(self):
        super().__init__('go_to_pose_server')
        self.declare_parameter('max_linear_speed', 2.0)
        self.declare_parameter('max_angular_speed', 2.0)
        self.declare_parameter('position_tolerance', 0.08)
        self.declare_parameter('angle_tolerance', 0.08)
        self.add_on_set_parameters_callback(self.validate_parameters)

        self.pose = None
        self.cmd_pub = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
        self.pose_sub = self.create_subscription(Pose, '/turtle1/pose', self.pose_callback, 10)
        self.speed_service = self.create_service(SetSpeed, '/set_speed', self.set_speed_callback)
        self.action_server = ActionServer(
            self, GoToPose, '/goToPose', execute_callback=self.execute_callback,
            goal_callback=self.goal_callback, cancel_callback=self.cancel_callback)
        self.get_logger().info('GoToPose action server ready on /goToPose')

    def validate_parameters(self, params):
        for param in params:
            if param.name in ('max_linear_speed', 'max_angular_speed',
                              'position_tolerance', 'angle_tolerance'):
                if param.value <= 0.0:
                    return SetParametersResult(successful=False,
                                               reason=f'{param.name} must be greater than zero')
        return SetParametersResult(successful=True)

    def pose_callback(self, msg):
        self.pose = msg

    def set_speed_callback(self, request, response):
        if request.max_linear_speed <= 0.0 or request.max_angular_speed <= 0.0:
            response.success = False
            response.message = 'Both speed limits must be greater than zero.'
            return response
        results = self.set_parameters([
            rclpy.parameter.Parameter('max_linear_speed', value=float(request.max_linear_speed)),
            rclpy.parameter.Parameter('max_angular_speed', value=float(request.max_angular_speed)),
        ])
        failed = [item for item in results if not item.successful]
        response.success = not failed
        response.message = ('Speed limits updated.' if not failed else
                            '; '.join(item.reason for item in failed if item.reason))
        return response

    def goal_callback(self, goal_request):
        p = goal_request.target_pose
        if not (0.0 <= p.x <= 11.0 and 0.0 <= p.y <= 11.0):
            self.get_logger().warning('Rejected target outside turtlesim bounds (0 to 11).')
            return GoalResponse.REJECT
        return GoalResponse.ACCEPT

    def cancel_callback(self, _goal_handle):
        return CancelResponse.ACCEPT

    def stop_turtle(self):
        self.cmd_pub.publish(Twist())

    async def execute_callback(self, goal_handle):
        result = GoToPose.Result()
        feedback = GoToPose.Feedback()
        target = goal_handle.request.target_pose
        rate = self.create_rate(30)
        timeout_seconds = 60.0
        elapsed = 0.0
        dt = 1.0 / 30.0
        try:
            while rclpy.ok():
                if goal_handle.is_cancel_requested:
                    self.stop_turtle()
                    goal_handle.canceled()
                    result.success = False
                    result.message = 'Goal canceled.'
                    return result
                if self.pose is None:
```

```
                    await rate.sleep()
                    elapsed += dt
                    if elapsed >= timeout_seconds:
                        break
                    continue

                dx = target.x - self.pose.x
                dy = target.y - self.pose.y
                distance = math.hypot(dx, dy)
                feedback.distance_remaining = distance
                goal_handle.publish_feedback(feedback)

                if distance <= self.get_parameter('position_tolerance').value:
                    yaw_error = normalize_angle(target.yaw - self.pose.theta)
                    if abs(yaw_error) <= self.get_parameter('angle_tolerance').value:
                        result.success = True
                        result.message = 'Target pose reached.'
                        goal_handle.succeed()
                        return result
                    cmd = Twist()
                    cmd.angular.z = max(-self.get_parameter('max_angular_speed').value,
                                        min(self.get_parameter('max_angular_speed').value,
                                            4.0 * yaw_error))
                else:
                    desired_heading = math.atan2(dy, dx)
                    heading_error = normalize_angle(desired_heading - self.pose.theta)
                    cmd = Twist()
                    cmd.angular.z = max(-self.get_parameter('max_angular_speed').value,
                                        min(self.get_parameter('max_angular_speed').value,
                                            4.0 * heading_error))
                    if abs(heading_error) < 0.6:
                        cmd.linear.x = min(self.get_parameter('max_linear_speed').value,
                                           1.5 * distance)
                self.cmd_pub.publish(cmd)
                await rate.sleep()
                elapsed += dt
                if elapsed >= timeout_seconds:
                    break
        except Exception as exc:  # ensure the turtle is stopped on unexpected errors
            self.get_logger().error(f'Action failed: {exc}')
            result.message = f'Action failed: {exc}'
        self.stop_turtle()
        if goal_handle.is_active:
            goal_handle.abort()
        result.success = False
        if not result.message:
            result.message = 'Timed out before reaching the target pose.'
        return result

    def destroy_node(self):
        self.stop_turtle()
        self.action_server.destroy()
        super().destroy_node()


def main(args=None):
    rclpy.init(args=args)
    node = GoToPoseServer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

How the main parts work

Imports

ROS action, node, geometry message, pose, parameter, and custom interface classes are imported so the node can subscribe, publish, accept goals, and update parameters.

Pose subscription

The `/turtle1/pose` callback stores the most recent turtle pose. The controller needs current x/y/yaw to calculate what to do next.

Action execution

The goal callback validates/accepts the request; the execution callback controls the turtle until it reaches the target or the task is cancelled/aborted.

Distance and angle errors

Distance is the straight-line distance to the target. Heading error is the difference between the desired direction and current yaw, normalized to avoid unnecessary full turns.

Feedback and result

The action publishes `distance_remaining` during motion and returns `success` plus a message at the end.

SetSpeed service

The service validates positive values and updates `max_linear_speed` and `max_angular_speed` using ROS parameter APIs.

8. Complete source code — Reset service server

This node provides `/reset_turtle`. It can call turtlesim's built-in `/reset` service to clear the drawing and uses `/turtle1/teleport_absolute` to return the turtle to the center with a zero heading.

reset_server.py — complete file

```
#!/usr/bin/env python3
"""Service server that optionally clears the screen and resets turtle pose."""
import asyncio

import rclpy
from rclpy.node import Node
from std_srvs.srv import Empty
from turtlesim.srv import TeleportAbsolute
from lab1_interfaces.srv import ResetTurtle


class ResetServer(Node):
    def __init__(self):
        super().__init__('reset_server')
        self.reset_client = self.create_client(Empty, '/reset')
        self.teleport_client = self.create_client(TeleportAbsolute, '/turtle1/teleport_absolute')
        self.service = self.create_service(ResetTurtle, '/reset_turtle', self.handle_reset)
        self.get_logger().info('ResetTurtle service ready on /reset_turtle')

    async def wait_for_service(self, client, name):
        while rclpy.ok() and not client.service_is_ready():
            self.get_logger().info(f'Waiting for {name}...')
            await asyncio.sleep(0.25)

    async def handle_reset(self, request, response):
        try:
            if request.clear_screen:
                await self.wait_for_service(self.reset_client, '/reset')
                future = self.reset_client.call_async(Empty.Request())
                await future

            await self.wait_for_service(self.teleport_client, '/turtle1/teleport_absolute')
            teleport = TeleportAbsolute.Request()
            teleport.x, teleport.y, teleport.theta = 5.544, 5.544, 0.0
            await self.teleport_client.call_async(teleport)
            response.success = True
            response.message = 'Screen cleared (if requested) and turtle reset to the center.'
        except Exception as exc:
            response.success = False
            response.message = f'Reset failed: {exc}'
            self.get_logger().error(response.message)
        return response


def main(args=None):
    rclpy.init(args=args)
    node = ResetServer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Important details

The custom service is separate from the built-in turtlesim services. The client must wait for the built-in service to become available before calling it. The response should clearly indicate whether the reset completed.

9. Complete source code — DrawSquare server

This action server accepts side length and a speed multiplier. It requests a reset, computes four corner poses, calls `/goToPose` once for each corner, and updates `sides_completed` feedback after each successful goal.

square_server.py — complete file

```
#!/usr/bin/env python3
"""DrawSquare action server; each side is completed by one GoToPose goal."""
import asyncio
import math

import rclpy
from rclpy.action import ActionClient, ActionServer, CancelResponse, GoalResponse
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node
from rcl_interfaces.srv import GetParameters, SetParameters
from rcl_interfaces.msg import Parameter, ParameterType, ParameterValue
from lab1_interfaces.action import DrawSquare, GoToPose
from lab1_interfaces.msg import TargetPose
from lab1_interfaces.srv import ResetTurtle


class SquareServer(Node):
    def __init__(self):
        super().__init__('square_server')
        self.go_to_pose_client = ActionClient(self, GoToPose, '/goToPose')
        self.reset_client = self.create_client(ResetTurtle, '/reset_turtle')
        self.get_params_client = self.create_client(GetParameters, '/go_to_pose_server/get_parameters')
        self.set_params_client = self.create_client(SetParameters, '/go_to_pose_server/set_parameters')
        self.action_server = ActionServer(
            self, DrawSquare, '/draw_square', execute_callback=self.execute_callback,
            goal_callback=self.goal_callback, cancel_callback=self.cancel_callback)
        self.get_logger().info('DrawSquare action server ready on /draw_square')

    def goal_callback(self, request):
        if request.side_length <= 0.0 or request.speed < 0.05 or request.speed > 1.0:
            return GoalResponse.REJECT
        if request.side_length > 4.5:
            return GoalResponse.REJECT
        return GoalResponse.ACCEPT

    def cancel_callback(self, _goal_handle):
        return CancelResponse.ACCEPT

    async def wait_ready(self, client, name):
        while rclpy.ok() and not client.wait_for_service(timeout_sec=0.25):
            self.get_logger().info(f'Waiting for {name}...')

    async def call_go_to_pose(self, x, y, yaw, goal_handle):
        if goal_handle.is_cancel_requested:
            return False, 'Square drawing canceled.'
        while not self.go_to_pose_client.wait_for_server(timeout_sec=0.25):
            await asyncio.sleep(0.1)
        goal = GoToPose.Goal()
        goal.target_pose = TargetPose(x=float(x), y=float(y), yaw=float(yaw))
        send_future = self.go_to_pose_client.send_goal_async(goal)
        await send_future
        handle = send_future.result()
        if handle is None or not handle.accepted:
            return False, 'GoToPose goal was rejected.'
        result_future = handle.get_result_async()
        await result_future
        wrapped = result_future.result()
        return wrapped.result.success, wrapped.result.message

    async def execute_callback(self, goal_handle):
        result = DrawSquare.Result()
        feedback = DrawSquare.Feedback()
        request = goal_handle.request
        original_speed = None
        try:
            await self.wait_ready(self.reset_client, '/reset_turtle')
            reset_future = self.reset_client.call_async(ResetTurtle.Request(clear_screen=True))
            await reset_future
            reset_result = reset_future.result()
            if not reset_result.success:
                raise RuntimeError(reset_result.message)

            # The action's speed is a multiplier in [0.05, 1.0]. Keep the base parameter
            # as the upper limit and restore it after the drawing operation.
            await self.wait_ready(self.get_params_client, 'GoToPose parameter service')
            get_future = self.get_params_client.call_async(
                GetParameters.Request(names=['max_linear_speed']))
            await get_future
            values = get_future.result().values
            if values and values[0].type == ParameterType.PARAMETER_DOUBLE:
                original_speed = values[0].double_value
            else:
                original_speed = 2.0
            effective_speed = min(original_speed, max(0.05, original_speed * request.speed))
            await self.wait_ready(self.set_params_client, 'GoToPose parameter service')
            param = Parameter(name='max_linear_speed', value=ParameterValue(
                type=ParameterType.PARAMETER_DOUBLE, double_value=effective_speed))
            set_future = self.set_params_client.call_async(SetParameters.Request(parameters=[param]))
            await set_future
            if not set_future.result().results[0].successful:
                raise RuntimeError('Could not apply requested drawing speed.')

            # Begin at the center and draw four equal sides, turning 90 degrees each time.
            start_x = start_y = 5.544
```

```
            side = float(request.side_length)
            # A square fits comfortably inside turtlesim for side lengths up to 4.5.
            # Use an axis-aligned path, starting from its lower-left corner.
            x0, y0 = start_x - side / 2.0, start_y - side / 2.0
            points = [(x0 + side, y0), (x0 + side, y0 + side),
                      (x0, y0 + side), (x0, y0)]
            yaw = 0.0
            for index, (x, y) in enumerate(points, start=1):
                if goal_handle.is_cancel_requested:
                    result.success = False
                    result.message = 'Square drawing canceled.'
                    goal_handle.canceled()
                    return result
                ok, message = await self.call_go_to_pose(x, y, yaw, goal_handle)
                if not ok:
                    raise RuntimeError(message)
                feedback.sides_completed = index
                goal_handle.publish_feedback(feedback)
            result.success = True
            result.message = 'Square completed using exactly four GoToPose goals.'
            goal_handle.succeed()
        except Exception as exc:
            result.success = False
            result.message = str(exc)
            self.get_logger().error(f'DrawSquare failed: {exc}')
            if goal_handle.is_active:
                goal_handle.abort()
        finally:
            if original_speed is not None:
                try:
                    restore = Parameter(name='max_linear_speed', value=ParameterValue(
                        type=ParameterType.PARAMETER_DOUBLE, double_value=original_speed))
                    restore_future = self.set_params_client.call_async(
                        SetParameters.Request(parameters=[restore]))
                    await restore_future
                except Exception as exc:
                    self.get_logger().warning(f'Could not restore linear speed parameter: {exc}')
        return result


def main(args=None):
    rclpy.init(args=args)
    node = SquareServer()
    executor = MultiThreadedExecutor(num_threads=4)
    executor.add_node(node)
    try:
        executor.spin()
    except KeyboardInterrupt:
        pass
    finally:
        executor.shutdown()
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

How DrawSquare works

- Validate the requested side length and speed multiplier before starting.
- Reset the turtle so each run starts from a known position.
- Calculate four corners around the center of the turtlesim window.
- For each corner, send a GoToPose goal and wait for its result.
- Abort if a side fails; otherwise increment and publish the completed-side count.
- Return a final success result only after all four goals finish.

10. Launch file and parameters

The launch file starts the simulator and all controller nodes. This lets you run the complete system with one command. The YAML file documents initial speed and tolerance values.

lab1.launch.py — complete file

```
from launch import LaunchDescription
from launch_ros.actions import Node


def generate_launch_description():
    return LaunchDescription([
        Node(package='turtlesim', executable='turtlesim_node', name='turtlesim_node', output='screen'),
        Node(package='lab1_controller', executable='go_to_pose_server', name='go_to_pose_server',
             parameters=[{'max_linear_speed': 2.0, 'max_angular_speed': 2.0,
                          'position_tolerance': 0.08, 'angle_tolerance': 0.08}], output='screen'),
        Node(package='lab1_controller', executable='reset_server', name='reset_server', output='screen'),
        Node(package='lab1_controller', executable='square_server', name='square_server', output='screen'),
    ])
```

params.yaml — complete file

```
go_to_pose_server:
  ros__parameters:
    max_linear_speed: 2.0
    max_angular_speed: 2.0
    position_tolerance: 0.08
    angle_tolerance: 0.08
```

Parameter inspection and update

Read and change parameters

```
ros2 param list /go_to_pose_server
ros2 param get /go_to_pose_server max_linear_speed
ros2 param get /go_to_pose_server max_angular_speed
ros2 param set /go_to_pose_server max_linear_speed 1.0
ros2 param set /go_to_pose_server max_angular_speed 1.5
```

11. Test every requirement

Run these tests from a second terminal after sourcing ROS and the workspace.

Check nodes and interfaces

```
ros2 node list
ros2 action list
ros2 service list
ros2 interface show lab1_interfaces/msg/TargetPose
ros2 interface show lab1_interfaces/action/GoToPose
ros2 interface show lab1_interfaces/action/DrawSquare
ros2 interface show lab1_interfaces/srv/ResetTurtle
ros2 interface show lab1_interfaces/srv/SetSpeed
```

Test ResetTurtle

```
ros2 service call /reset_turtle lab1_interfaces/srv/ResetTurtle "{clear_screen: true}"
```

Expected: a successful response and the turtle returns to the center.

Test SetSpeed

```
ros2 service call /set_speed lab1_interfaces/srv/SetSpeed "{max_linear_speed: 1.0, max_angular_speed: 1.5}"
```

Expected: success true; then confirm the parameter values using `ros2 param get`.

Test GoToPose

```
ros2 action send_goal /goToPose lab1_interfaces/action/GoToPose \
"{target_pose: {x: 8.0, y: 8.0, yaw: 0.0}}" --feedback
```

Expected: distance feedback while moving, followed by a result. Keep target coordinates inside the turtlesim area (approximately 0–11).

Test DrawSquare

```
ros2 action send_goal /draw_square lab1_interfaces/action/DrawSquare \
"{side_length: 2.0, speed: 0.5}" --feedback
```

Expected: `sides_completed` feedback values 1, 2, 3, and 4 and then success true.

12. Troubleshooting

Package not found

Source `/opt/ros/jazzy/setup.bash` and `~/ros2_ws/install/setup.bash`. Ensure both packages are directly inside `~/ros2_ws/src/`, then rebuild.

Custom interface import error

Rebuild after editing `.msg`, `.srv`, or `.action` files, then source the workspace in the terminal running the node.

Service unavailable

Check that launch is running and the exact service name/type is correct using `ros2 service list` and `ros2 service type /set_speed`.

Turtlesim window not visible

Check WSLg/display support and whether the window is behind another application.

Action does not finish

Inspect the launch terminal for exceptions, ensure pose updates are arriving, and avoid launching duplicate stacks.

Square has small gaps

A simulated closed loop uses sampled pose data and tolerances. Check action result and feedback, not only the drawn line.

Build fails after interface changes

Run `colcon build --symlink-install` again and source `install/setup.bash` after a successful build.

13. Viva questions and answers

What is ROS 2?

A middleware framework for building distributed robot applications from nodes that communicate through topics, services, and actions.

Why use a custom message?

To define a reusable data structure that fits the project's target-pose information.

What are the three parts of an action?

Goal, result, and feedback, separated by `---` lines.

Why use an action instead of a service for movement?

Movement is long-running and benefits from progress feedback and cancellation.

What does `/turtle1/cmd_vel` carry?

A `geometry_msgs/msg/Twist` message with linear and angular velocity commands.

What does `/turtle1/pose` provide?

Current turtle x/y position and orientation.

What is the purpose of ResetTurtle?

To optionally clear the trail and return the turtle to a known pose.

Why does DrawSquare call GoToPose four times?

It reuses the tested motion controller for each of the square's four corners.

What is action feedback?

Intermediate progress data sent before the final result.

What does `/set_speed` do?

Changes the maximum linear and angular speed parameters while the node is running.

Why use a launch file?

To start multiple nodes consistently from one command.

What is colcon?

The build tool commonly used to build ROS 2 workspaces.

Why source install/setup.bash?

It updates environment variables so ROS 2 can discover the built packages and generated interfaces.

14. GitHub publishing

Before publishing, personalize maintainer information, run a clean build, test the services/actions, and add genuine screenshots if your instructor asks for them. Do not upload build outputs, logs, credentials, or screenshots you did not capture.

Git commands

```
git init
git add .
git status
git commit -m "Add ROS 2 Jazzy Lab 1 project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/ros2-jazzy-lab1.git
git push -u origin main
```

Replace `YOUR-USERNAME` with your actual GitHub username. If you already cloned the remote repository, work inside that cloned directory and do not run `git init` again.

15. Final submission checklist

- ■ Both packages are in the correct workspace structure.
- ■ Maintainer name/email are personalized.
- ■ A clean colcon build completes successfully.
- ■ Launch starts the simulator and all three controller nodes.
- ■ ResetTurtle service returns success.
- ■ SetSpeed service updates both speed parameters.
- ■ GoToPose reaches a valid target and returns success.
- ■ DrawSquare reports exactly four completed sides and success.
- ■ README includes build, launch, and test instructions.
- ■ GitHub repository opens and files/README render correctly.

Appendix A. Package metadata and README

The files below are part of the repository and define build configuration, dependencies, entry points, and project instructions. The full source files are included in the PDF so the document remains useful as a code reference; the repository is still the best place to edit the code.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(lab1_interfaces)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(geometry_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/TargetPose.msg"
  "action/GoToPose.action"
  "action/DrawSquare.action"
  "srv/ResetTurtle.srv"
  "srv/SetSpeed.srv"
  DEPENDENCIES geometry_msgs
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

package.xml (interfaces)

```
<?xml version="1.0"?>
<package format="3">
  <name>lab1_interfaces</name>
  <version>0.0.1</version>
  <description>Custom messages, actions, and services for ROS 2 Lab 1.</description>
  <maintainer email="student@example.com">Student</maintainer>
  <license>MIT</license>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <build_depend>rosidl_default_generators</build_depend>
  <depend>geometry_msgs</depend>
  <exec_depend>rosidl_default_runtime</exec_depend>
  <member_of_group>rosidl_interface_packages</member_of_group>
  <export><build_type>ament_cmake</build_type></export>
</package>
```

package.xml (controller)

```
<?xml version="1.0"?>
<package format="3">
  <name>lab1_controller</name>
  <version>0.0.1</version>
  <description>ROS 2 Jazzy turtlesim controller, action servers, and reset service.</description>
  <maintainer email="student@example.com">Student</maintainer>
  <license>MIT</license>
  <buildtool_depend>ament_python</buildtool_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>rcl_interfaces</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>turtlesim</exec_depend>
  <exec_depend>std_srvs</exec_depend>
  <exec_depend>lab1_interfaces</exec_depend>
  <exec_depend>launch</exec_depend>
  <exec_depend>launch_ros</exec_depend>
  <export><build_type>ament_python</build_type></export>
</package>
```

setup.py

```
from setuptools import find_packages, setup
from glob import glob
import os

package_name = 'lab1_controller'

setup(
    name=package_name,
    version='0.0.1',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
        (os.path.join('share', package_name, 'config'), glob('config/*.yaml')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='ROS 2 Jazzy turtlesim action and service lab.',
    license='MIT',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'go_to_pose_server = lab1_controller.go_to_pose_server:main',
            'reset_server = lab1_controller.reset_server:main',
            'square_server = lab1_controller.square_server:main',
        ],
    },
)
```

README.md

```
# ROS 2 Jazzy Lab 1 — Turtlesim, Actions & Services

A ROS 2 Jazzy / Ubuntu 24.04 project demonstrating custom interfaces, action servers, a service server, runtime parameters, and launch-based startup us

> **Before submission:** replace the generic maintainer information in both `package.xml` files and `setup.py` with your own details. Run the verificat

## Features

- `lab1_interfaces`: custom `TargetPose` message, `GoToPose` and `DrawSquare` actions, and `ResetTurtle` service.
- `/goToPose`: drives the turtle to a target `(x, y, yaw)` and publishes remaining-distance feedback.
- `/reset_turtle`: optionally clears the drawing and teleports the turtle to the center.
- `/draw_square`: draws four sides by sending exactly four `GoToPose` goals; feedback reports completed sides.
- Runtime parameters for linear/angular speed and pose tolerances.
- One launch file starts all four nodes.

## Requirements

- Ubuntu 24.04 (native or WSL2 with a working graphical display)
- ROS 2 Jazzy
- `turtlesim`, `colcon`, and `rosdep`

Install dependencies if needed:

```bash
sudo apt update
sudo apt install -y ros-jazzy-turtlesim python3-colcon-common-extensions python3-rosdep
```

Build

Open a terminal and run:

```bash
source /opt/ros/jazzy/setup.bash
mkdir -p ~/ros2_ws/src
# Copy this repository's two package folders into ~/ros2_ws/src/
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

If this repository itself is cloned inside `~/ros2_ws/src`, the package folders are already in the correct place. Do not put the repository root one le

Run everything

```bash
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch lab1_controller lab1.launch.py
```

Keep that terminal open. The turtlesim window should appear, and the log should show the three controller servers becoming ready.

Test the service

In a **second terminal**:

```bash
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 service call /reset_turtle lab1_interfaces/srv/ResetTurtle "{clear_screen: true}"
```

Expected response: `success: true` with a message confirming reset.

Test GoToPose

```bash
ros2 action send_goal /goToPose lab1_interfaces/action/GoToPose \
"{target_pose: {x: 8.0, y: 8.0, yaw: 0.0}}" --feedback
```

The action should provide `distance_remaining` feedback and finish with a success result when the target pose is reached. Valid x/y coordinates are wi

Test DrawSquare

```bash
ros2 action send_goal /draw_square lab1_interfaces/action/DrawSquare \
"{side_length: 2.0, speed: 0.5}" --feedback
```

- `side_length`: side length in turtlesim coordinates; use a modest value such as `2.0`.
- `speed`: multiplier from `0.05` to `1.0`; lower values draw more slowly, and `1.0` uses the configured maximum linear speed.
- Feedback should report `sides_completed` as 1, 2, 3, and 4.

Inspect parameters

```bash
ros2 param list /go_to_pose_server
ros2 param get /go_to_pose_server max_linear_speed
ros2 param set /go_to_pose_server max_linear_speed 1.0
ros2 param set /go_to_pose_server max_angular_speed 1.5
```
```

```
Parameter values must be greater than zero.

## Package structure

```text
ros2_lab1_github/
■■■ lab1_interfaces/
■   ■■■ action/GoToPose.action
■   ■■■ action/DrawSquare.action
■   ■■■ msg/TargetPose.msg
■   ■■■ srv/ResetTurtle.srv
■   ■■■ CMakeLists.txt
■   ■■■ package.xml
■■■ lab1_controller/
■   ■■■ lab1_controller/
■   ■   ■■■ go_to_pose_server.py
■   ■   ■■■ reset_server.py
■   ■   ■■■ square_server.py
■   ■■■ launch/lab1.launch.py
■   ■■■ config/params.yaml
■   ■■■ setup.py
■   ■■■ package.xml
■■■ docs/
■■■ README.md
■■■ LICENSE
```

Interface definitions

`TargetPose.msg`

```text
float64 x
float64 y
float64 yaw
```

`GoToPose.action`

```text
lab1_interfaces/TargetPose target_pose
---
bool success
string message
---
# ... see the complete file in the repository ...
```

Final note: use the exact repository source files as the source of truth. This PDF includes the source files found in the prepared repository; if you changed a file locally after downloading the ZIP, regenerate or update the guide to match those changes.
```